Rozbudowa i ewaluacja rozwiązania Multi-Agent LLMs do analizy sprawozdań finansowych

Maciej Matuszewski, Marcin Bagnowski

8 kwietnia 2026

Spis treści

1 Aktualny stan projektu **2**
1.1 Kontekst i cel systemu 2
1.2 Architektura wieloagentowa i tryby działania 3
1.2.1 Tryb sekwencyjny (Sequential) 3
1.2.2 Tryb debaty (Group Chat) 3
1.3 Zintegrowany system automatycznej ewaluacji 4
1.4 Wnioski z dotychczasowych ewaluacji i ograniczenia systemu 4

2 Analiza literaturowa architektur multi-agent LLM **6**
2.1 Architektury typu concurrent 6
2.2 Architektury typu parallel 8
2.2.1 Przegląd literatury 8
2.2.2 Charakterystyka podejścia 8

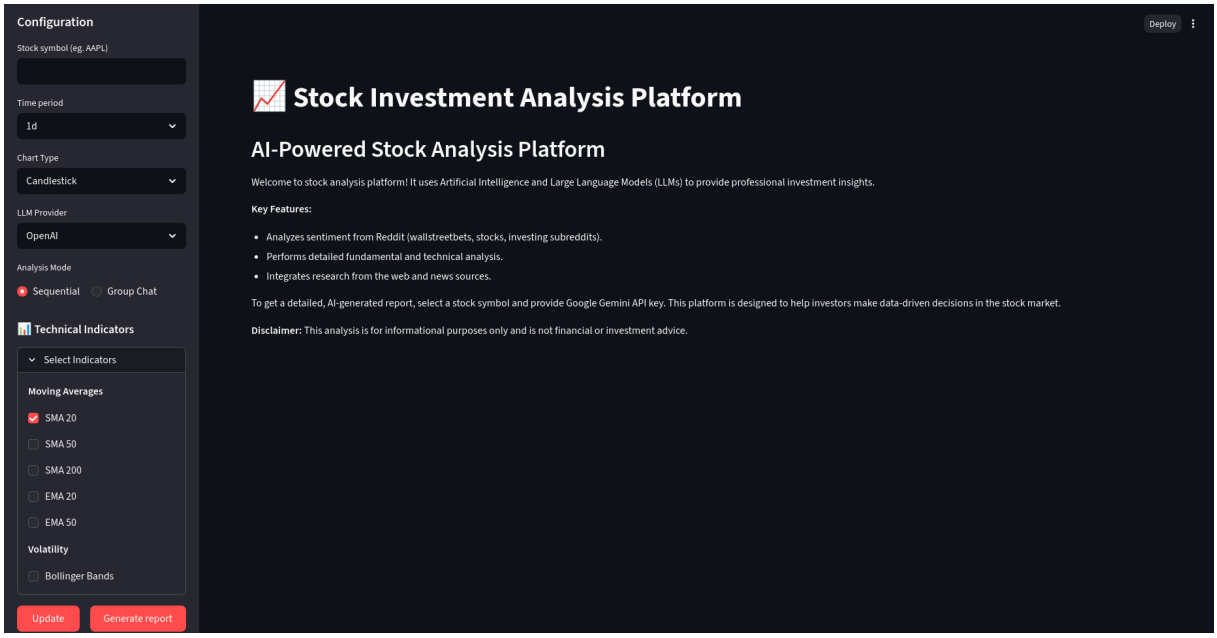
3 Rozbudowa projektu **9**
3.1 Optymalizacja trybu Group Chat 9
3.1.1 Current Context Storage 9
3.1.2 RAG sesyjny jako pamięć dowodów 10
3.1.3 Nowy model debaty agentów 11
3.2 Tryb współpracy Concurrent 11
3.3 Tryb współpracy Parallel 12

1 Aktualny stan projektu

1.1 Kontekst i cel systemu

Niniejszy projekt stanowi kolejną iterację rozwoju platformy inwestycyjnej opartej na frameworku CrewAI oraz modelach językowych (LLM). Odziedziczony przez nasz zespół system, rozwijany w poprzednich etapach pod nazwą *Rozbudowa projektu Systemy Multi-Agentowe LLM do Analizy Sprawozdań Finansowych* [1], jest w pełni funkcjonalnym narzędziem analitycznym.

Głównym celem zastanego systemu jest automatyczne generowanie kompleksowych raportów inwestycyjnych dla wybranych spółek giełdowych. Platforma skutecznie łączy analizę ilościową (wskaźniki techniczne, dane fundamentalne) z analizą jakościową (sentyment rynkowy, informacje z mediów). Interfejs użytkownika został zrealizowany w technologii Streamlit, co pozwala na wygodne wprowadzanie symboli giełdowych, wybór dostawcy modelu LLM (Google Gemini lub OpenAI) oraz konfigurację parametrów wizualizacji wskaźników technicznych (np. SMA, EMA, Wstęgi Bollingera). System posiada również wbudowaną funkcję eksportu wygenerowanych raportów wraz z wykresami do formatu PDF. Na rysunku 1 przedstawiono ekran startowy aplikacji.



Rysunek 1: Ekran startowy aplikacji prezentujący interfejs użytkownika.

W warstwie integracji danych system opiera się na stabilnych i profesjonalnych interfejsach API: Yahoo Finance, Finnhub API oraz Alpha Vantage API, które w poprzedniej iteracji zastąpiły problematyczne źródła oparte na serwisie Reddit.

1.2 Architektura wieloagentowa i tryby działania

Zastana architektura skutecznie adresuje typowe problemy systemów opartych na pojedynczym modelu LLM, takie jak trudność w integracji heterogenicznych danych czy brak krytycznej oceny własnych wniosków. System pozwala na realizację procesu analitycznego w dwóch odrębnych trybach współpracy agentów:

1.2.1 Tryb sekwencyjny (Sequential)

W tym trybie system funkcjonuje jako klasyczny, liniowy przepływ przetwarzania danych. Każdy agent wykonuje swoje zadanie i przekazuje ustrukturyzowane wyniki za pośrednictem centralnego koordynatora (`StockAnalysisCrew`), do kolejnego agenta w łańcuchu. W skład zespołu wchodzą cztery wyspecjalizowane role:

- **Senior Stock Market Researcher** – odpowiedzialny za gromadzenie danych jakościowych oraz analizę sentymentu na podstawie wiadomości rynkowych.
- **Expert Technical Analyst** – przeprowadzający analizę techniczną kursu akcji z wykorzystaniem biblioteki TA-Lib (m.in. wskaźniki RSI, MACD, Wstęgi Bollingera).
- **Senior Fundamental Analyst** – badający kondycję finansową spółki na podstawie kluczowych wskaźników (P/E, P/S, D/E, ROE).
- **Chief Investment Strategist (Reporter)** – integrujący wnioski z poprzednich etapów w spójny, końcowy raport inwestycyjny.

1.2.2 Tryb debaty (Group Chat)

Alternatywny tryb, inspirowany architekturą FinDebate [2], opiera się na dwuetapowym procesie przetwarzania informacji, w którym łącznie bierze udział sześciu wyspecjalizowanych agentów.

W pierwszej fazie (**analizy specjalistycznej**) działają agenci domenowi znani z trybu sekwencyjnego: *Senior Stock Market Researcher*, *Expert Technical Analyst* oraz *Senior Fundamental Analyst*. Ich zadaniem jest równoległe zgromadzenie surowych danych rynkowych, obliczenie wskaźników i przygotowanie wstępnych wniosków.

Po zakończeniu tej fazy system uruchamia drugą fazę – **debatę międzyagentową**. Biorą w niej udział trzy nowe meta-role, które nie pobierają danych bezpośrednio z API, lecz operują na wynikach dostarczonych przez analityków:

- **Agent Sceptyk (Sceptic)** – którego zadaniem jest krytyczna analiza, wskazywanie nieścisłości, słabych punktów argumentacji i potencjalnych ryzyk.
- **Agent Zaufania (Trust)** – weryfikujący wiarygodność danych i uzasadnień, wzmacniający pozytywne argumenty.

- **Agent Lider (Leader)** – pełniący funkcję moderatora dyskusji, który ostatecznie integruje ścierające się perspektywy w zbalansowany raport końcowy.

Taka struktura pozwala na wprowadzenie elementu wewnętrznej weryfikacji i naturalne zderzenie perspektywy byczej (*bullish*) z niedźwiedzią (*bearish*).

1.3 Zintegrowany system automatycznej ewaluacji

Kluczowym komponentem odziedziczonym po poprzedniej iteracji projektu jest autorski, w pełni zautomatyzowany system oceny jakości generowanych raportów. System ten, zintegrowany bezpośrednio z interfejsem Streamlit, pozwala na deterministyczną ocenę raportów bez konieczności manualnej interwencji. Ewaluacja opiera się na pięciu wymiarach:

1. **Struktura dokumentu (waga: 25%)** – weryfikuje obecność 8 wymaganych sekcji tematycznych (m.in. Executive Summary, Risk Assessment) oraz zlicza elementy strukturalne, takie jak nagłówki i paragrafy.
2. **Bogactwo danych (waga: 25%)** – mierzy gęstość informacji poprzez automatyczną ekstrakcję (za pomocą wyrażeń regularnych) konkretnych wartości liczbowych, 38 zdefiniowanych wskaźników finansowych oraz cytowań źródeł.
3. **Profesjonalizm języka (waga: 20%)** – implementuje odwrócony algorytm Flesch Reading Ease do pomiaru wyrafinowania językowego tekstu na podstawie długości zdań i liczby sylab.
4. **Użyteczność inwestycyjna (waga: 20%)** – weryfikuje obecność jasnej rekomendacji (Buy/Sell/Hold) popartej konkretnymi parametrami: ceną docelową (*price target*) oraz horyzontem czasowym.
5. **Balans sentymentu (waga: 10%)** – wykorzystuje specjalistyczne słowniki finansowe (inspirowane badaniami Loughran-McDonald) do obliczenia stosunku słów o zabarwieniu pozytywnym i negatywnym.

1.4 Wnioski z dotychczasowych ewaluacji i ograniczenia systemu

Z obszernej dokumentacji i wyników zautomatyzowanej ewaluacji z poprzedniej iteracji [1] wynika, że oba tryby działania wykazują skrajnie odmienne charakterystyki. Ostatecznym wnioskiem poprzedniego zespołu było uznanie **trybu sekwencyjnego za obecnie lepszy i bardziej efektywny wybór** do generowania profesjonalnych raportów inwestycyjnych (średni wynik ogólny 82,2/100 wobec 69,3/100 dla trybu Group Chat).

Szczegółowa analiza porównawcza ujawniła jednak **wyraźne zalety architektury debatowej (Group Chat)**, która wykazuje przewagę w zastosowaniach niszowych i aka-

demickich:

- **Profesjonalizm języka (Professional Sophistication):** Proces debaty prowadzi do wieloetapowej rafinacji języka. Agenci debatowi "uszlachetniają"styl wypowiedzi, co skutkuje wysoce formalnym, akademickim tekstem (średni wynik 91,8/100 wobec 77,0/100 dla trybu sekwencyjnego).
- **Balans sentymentu (Sentiment Balance):** Dzięki obecności Agenta Sceptyka, raporty z trybu Group Chat charakteryzują się znacznie lepszym obiektywizmem i wyważeniem argumentów "za"i "przeciw"(średni wynik 76,2/100 vs 62,7/100).

Jednocześnie zdiagnozowano **krytyczne wady i ograniczenia trybu Group Chat**, które dyskwalifikują go z obecnych zastosowań produkcyjnych i stanowią główne wyzwanie architektoniczne dla dalszego rozwoju platformy:

- **Dramatyczny spadek bogactwa danych (Data Richness):** Średni wynik w tym wymiarze wyniósł zaledwie 45,8/100 (wobec 76,3/100 dla trybu sekwencyjnego). Poprzedni zespół zidentyfikował trzy główne przyczyny tej degradacji:
  1. *Kompresja informacji* – agenci debatowi skupiają się na agregacji argumentów, pomijając konkretne dane liczbowe wypracowane w pierwszej fazie.
  2. *Krótsze raporty* – teksty generowane w trybie debaty są średnio o 43% krótsze.
  3. *Brak dostępu do źródeł* – meta-agenci operują jedynie na syntezie, nie mając bezpośredniego dostępu do surowych danych z API.
- **Niestabilność rekomendacji (Actionability):** Tryb Group Chat wykazuje ekstremalną wariancję w generowaniu użytecznych wniosków (średnio 48,3/100 vs 100/100 w trybie sekwencyjnym). W imię "obiektywizmu", agenci często usuwają z finalnego raportu konkretne parametry, takie jak cena docelowa (*price target*) czy horyzont czasowy.
- **Niska efektywność czasowa:** Proces wieloagentowej dyskusji jest niezwykle kosztowny obliczeniowo – czas generowania raportu jest średnio 4,5-krotnie dłuższy niż w przypadku przetwarzania sekwencyjnego.

Jak zaznaczono w dokumentacji stanu zastanego, rozwiązanie problemu utraty danych liczbowych w fazie debatowej (poprawa mechanizmu ekstrakcji i retencji danych) jest warunkiem koniecznym, aby architektura Group Chat mogła spełniać standardy profesjonalnych raportów inwestycyjnych.

2 Analiza literaturowa architektur multi-agent LLM

Systemy multi-agent oparte na dużych modelach językowych (LLM) stanowią istotne rozszerzenie klasycznych systemów rozproszonych, jednak ich rola wykracza poza tradycyjne paradygmaty przetwarzania obliczeniowego. Współczesne badania wskazują, że architektury te mogą być interpretowane jako modele poznawcze, w których inteligencja wyłania się z interakcji wielu prostszych komponentów [3].

Szczególnie istotnym kierunkiem jest traktowanie systemów multi-agent jako realizacji koncepcji *Society of Mind*, gdzie procesy poznawcze nie są zlokalizowane w pojedynczym modelu, lecz powstają w wyniku współpracy wielu wyspecjalizowanych jednostek. Przykładowo, framework Concurrent Modular Agent pokazuje, że intencje oraz zachowania systemu mogą wyłaniać się z interakcji językowych między modułami działającymi asynchronicznie [4]. W kontekście analizy finansowej, gdzie konieczna jest jednoczesna synteza twardych danych rynkowych i miękkiego sentymentu medialnego, takie podejście pozwala na znacznie wierniejsze odwzorowanie pracy rzeczywistych zespołów analitycznych.

W literaturze współistnieją dwa ważne podejścia architektoniczne: **concurrent** (współbieżne) oraz **parallel** (równoległe). Różnią się one nie tylko sposobem organizacji obliczeń, lecz przede wszystkim mechanizmami powstawania wiedzy i podejmowania decyzji.

2.1 Architektury typu concurrent

W literaturze nowoczesnych systemów multi-agent LLM jednym z wyróżnianych paradygmatów jest podejście **concurrent**, w którym wiele agentów działa jednocześnie, a ich zachowanie i interakcje są koordynowane w czasie rzeczywistym. W przeciwieństwie do klasycznych sekwencyjnych przepływów wykonania, architektury concurrent dążą do osiągnięcia emergentnych właściwości poprzez współbieżną interakcję wielu autonomicznych modułów.

Przykładem takiej architektury jest *Concurrent Modular Agent (CMA)*, zaproponowany przez Maruyamę i in. [4]. Autorzy opisują framework, którego poszczególne moduły funkcjonują współbieżnie i niezależnie, a ich działanie jest częściowo skoordynowane poprzez wspólną logikę i mechanizmy komunikacji [4].

W architekturze CMA kluczowe elementy to:

- **Asynchroniczne działanie agentów** – każdy moduł wykonuje się niezależnie, bez globalnej blokady czy centralnego zegara, co tworzy środowisko współbieżne.
- **Komunikacja między agentami** – agenci wymieniają informacje werbalnie lub poprzez pomocnicze mechanizmy (np. wspólny stan) w celu koordynacji działań i wspólnej adaptacji do kontekstu.

- **Współdzielony stan globalny** – CMA wykorzystuje wspólny kontekst, który jest czytelny i aktualizowany przez wiele komponentów, co ułatwia synchronizację i utrzymanie spójności modeli mentalnych agentów.

Dzięki tym mechanizmom architektura CMA realizuje również koncepcję *Society of Mind*, gdzie złożona inteligencja może wyłonić się z interakcji prostszych procesów [4].

Innym przykładem pracy, która analizuje konkretne konsekwencje koordynacji agentów w podejściach współbieżnych, jest praca Sergey'a Pugacheva pt. *CodeCRDT: Observation-Driven Coordination for Multi-Agent LLM Code Generation* [5]. Chociaż głównym celem tej pracy jest formalizacja obserwacyjnej koordynacji agentów w kontekście generowania kodu, jej wyniki dostarczają ważnych obserwacji na temat zachowania agentów w środowisku concurrent. Autor pokazuje, że agenci mogą koordynować się poprzez monitorowanie uaktualnień we wspólnym stanie, zamiast poprzez bezpośrednie przesyłanie komunikatów [5]. Takie podejście umożliwia lock-free i conflict-free współbieżne wykonywanie zadań, ale jednocześnie ujawnia różnorodne wzorce zachowań i trade-offs wynikające z interakcji agentów w trybie współbieżnym.

Empiryczne badania przeprowadzone w tej pracy wykazały, że choć w niektórych zadaniach współbieżna koordynacja może prowadzić do przyspieszeń (np. do 21.1% poprawy czasu wykonania w porównaniu z sekwencyjnym podejściem), to w innych sytuacjach takie środowisko może nawet spowalniać działanie systemu (do 39.4% wolniej), co jest konsekwencją zachowań agentów i ich wzajemnej koordynacji [5]. Ponadto autor raportuje, że systemy te ujawniają także konflikty semantyczne na poziomie 5–10%, co oznacza, że wieloagentowa współpraca nie zawsze prowadzi do jednoznacznie lepszych rezultatów [5].

Podsumowując, architektury concurrent w systemach multi-agent LLM charakteryzują się:

- współbieżnym wykonywaniem autonomicznych agentów,
- intensywną koordynacją działań poprzez komunikację i wspólny stan,
- emergentnymi wzorcami zachowań, których jakość i efektywność są uzależnione od struktury zadań oraz mechanizmów koordynacyjnych.

Modele takie oferują potencjał do realizacji bardziej elastycznych i adaptacyjnych systemów AI, ale wymagają starannego zaprojektowania mechanizmów koordynacji i kontroli, aby uniknąć negatywnych efektów skali i konfliktów semantycznych.

2.2 Architektury typu parallel

2.2.1 Przegląd literatury

Współczesne badania nad paradygmatem równoległym (parallel) skupiają się na eliminacji wąskich gardeł komunikacyjnych oraz poprawie rzetelności danych poprzez izolację agentów:

- **Li et al. (2024) – *More Agents Is All You Need* [6]**
  Wprowadza metodę *Agent Forest*, opartą na instancjonowaniu wielu agentów rozwiązujących to samo zadanie. Ostateczny wynik ustalany jest poprzez głosowanie większościowe (*majority voting*). Autorzy wykazali, że zespół mniejszych modeli działających równolegle może przewyższyć skutecznością pojedynczy, znacznie większy model (np. GPT-4).
- **Anthropic (2025) – *How we built our multi-agent research system* [7]**
  Opisuje wzorzec *Orchestrator-Worker*, gdzie agent lider dekomponuje złożone zapytanie na niezależne podzadania delegowane do sub-agentów. Taka struktura zapobiega dekoncentracji modeli przez nadmiarowy kontekst, co znacząco redukuje ryzyko halucynacji w zadaniach badawczych.
- **M1-Parallel Framework (2025) – *Optimizing Sequential Multi-Step Tasks with Parallel LLM Agents* [8]**
  Proponuje zrównoleglenie całych zespołów realizujących wieloetapowe rozumowanie. Kluczowym wnioskiem jest możliwość sterowania wydajnością poprzez tryby *early termination* (priorytet szybkości) lub *aggregation* (priorytet jakości).
- **Xiao et al. (2025) – *TradingAgents* [9]**
  Prezentuje system finansowy, w którym agenci domenowi (fundamentalni, techniczni, sentymentu) analizują strumienie danych bez wzajemnej komunikacji. Takie podejście gwarantuje, że do końcowej syntezy trafiają dane „czyste" – niespaczone sugestiami innych agentów, co jest kluczowe w eliminowaniu rynkowych uprzedzeń (biases).

2.2.2 Charakterystyka podejścia

Architektura **parallel** realizuje koncepcję **równoległości poznawczej**. W przeciwieństwie do trybów konwersacyjnych, agenci pracują w pełnej izolacji, a ich wyniki są integrowane dopiero w fazie końcowej syntezy. Podejście to wyróżnia się następującymi cechami:

1. **Dekompozycja i specjalizacja:** Zadania są dzielone na mniejsze, atomowe fragmenty. Agent skupiony wyłącznie na jednym aspekcie (np. wskaźnikach technicznych) nie jest obciążony poznawczo danymi z innych dziedzin (np. sentymentem z mediów społecznościowych).

2. **Determinizm czasowy:** Całkowity czas wykonania fazy analitycznej ograniczony jest przez czas pracy najwolniejszego agenta (bottleneck), a nie sumę czasów wszystkich jednostek, co pozwala na drastyczną redukcję opóźnień (*latency reduction*).
3. **Eliminacja efektu domina:** W architekturach Parallel nie występuje ryzyko, że błąd lub błędna interpretacja jednego agenta wpłynie na tok rozumowania pozostałych przed zakończeniem ich pracy. Pozwala to na uzyskanie kilku niezależnych perspektyw, które dopiero agent-lider musi zważyć i połączyć.

Głównym wyzwaniem pozostaje konstrukcja mechanizmu agregacji, który musi rozstrzygać konflikty między sprzecznymi wynikami dostarczonymi przez niezależnie pracujące moduły.

3 Rozbudowa projektu

Niniejszy rozdział opisuje wprowadzone rozszerzenia systemu, które poprawiają jakość podejścia Group Chat oraz wprowadzają dwa nowe paradygmaty współpracy agentów. Wszystkie nowe tryby bazują na sprawdzonym zespole trzech agentów technicznych: *Senior Stock Market Researcher*, *Expert Technical Analyst* oraz *Senior Fundamental Analyst*. Dodatkowo, w trybie Parallel, wprowadzono czwartego agenta – **Reportera (Agregatora)** – uruchamianego wyłącznie w fazie syntezy wyników.

3.1 Optymalizacja trybu Group Chat

W poprzedniej iteracji systemu zidentyfikowano istotny problem utraty informacji (*information loss*) w trakcie debaty między agentami. Wynikało to z faktu, że komunikacja między agentami opierała się wyłącznie na warstwie językowej (history of chat), co prowadziło do fragmentacji wiedzy oraz ograniczonej możliwości śledzenia źródeł wniosków.

W celu rozwiązania tego problemu wprowadzono dwa komplementarne mechanizmy: **Current Context Storage** jako ustrukturyzowaną pamięć roboczą oraz **RAG sesyjny** jako mechanizm dostępu do dowodów wykorzystywanych w analizie. Mechanizmy te zostały pierwotnie zaprojektowane z myślą o trybie Group Chat, lecz – jak opisano w kolejnych podrozdziałach – stanowią również opcjonalne wsparcie dla trybów Concurrent i Parallel.

3.1.1 Current Context Storage

Wprowadzono **Current Context Storage**, czyli współdzieloną strukturę danych w formacie JSON, która pełni rolę centralnej pamięci roboczej dla całego systemu. W przeciwieństwie do poprzedniego podejścia, pamięć ta nie przechowuje jedynie wyników końcowych, lecz zawiera ustrukturyzowane elementy procesu analitycznego, takie jak:

- **facts** – twarde dane liczbowe oraz obserwacje wyekstrahowane ze źródeł (np. wskaźniki finansowe),
- **claims** – wnioski i interpretacje generowane przez agentów,

Pamięć ma charakter **append-only**, co oznacza, że agenci nie nadpisują istniejących danych, lecz **dopisują nowe wpisy** wraz z metadanymi (agent, odniesienia do źródeł, opcjonalne powiązanie z wcześniejszym wpisem). Pozwala to na zachowanie pełnej historii procesu decyzyjnego oraz umożliwia analizę konfliktów i ewolucji wniosków. Gdy agent chce wyrazić korektę lub sprzeciw wobec wcześniejszego twierdzenia, dodaje nowy wpis z polem `refutes_id` wskazującym na korygowany rekord – nie modyfikuje istniejących danych.

W nowym podejściu komunikacja między agentami odbywa się pośrednio poprzez aktualizację tej struktury, a nie wyłącznie poprzez generowanie tekstu. Takie podejście eliminuje problem utraty informacji oraz znacząco zwiększa przejrzystość procesu wnioskowania.

3.1.2 RAG sesyjny jako pamięć dowodów

W przeciwieństwie do klasycznych implementacji RAG, w których budowana jest trwała baza wiedzy, zastosowano podejście **RAG sesyjnego** (*session-scoped retrieval*). Mechanizm ten nie służy do przechowywania ogólnej wiedzy o spółkach, lecz do utrzymywania zbioru źródeł wykorzystanych w konkretnej analizie.

Każdy agent, pobierając dane z zewnętrznych API (np. Finnhub, Alpha Vantage), zapisuje istotne fragmenty źródeł w bazie wektorowej powiązanej z bieżącą sesją analityczną. Każdy zapis zawiera metadane, takie jak:

- identyfikator źródła (*source_id*),
- data publikacji,
- powiązanie z analizowaną spółką.

Dzięki temu:

- agenci w trakcie debaty mogą dynamicznie odwoływać się do tych samych źródeł,
- możliwa jest weryfikacja twierdzeń na podstawie konkretnych fragmentów danych (*evidence-based reasoning*),
- unika się problemu „starzenia się danych", ponieważ retrieval ograniczony jest do bieżącej sesji analitycznej.

RAG pełni więc rolę **pamięci dowodów**, a nie trwałej bazy wiedzy. Pozwala to na zachowanie spójności czasowej analizy oraz eliminuje ryzyko wpływu nieaktualnych informacji na końcowe wnioski. Należy zaznaczyć, że określenie *RAG sesyjny* odnosi się tutaj do wzorca dostępu do danych (retrieve-then-generate w obrębie sesji), a nie do budowy trwa-

łego indeksu wiedzy.

3.1.3 Nowy model debaty agentów

Wprowadzenie powyższych mechanizmów zmienia sposób interakcji agentów w trybie Group Chat. Zamiast klasycznej wymiany komunikatów tekstowych, proces debaty przyjmuje formę iteracyjnej aktualizacji wspólnego stanu:

1. Agenci analityczni zapisują fakty i wnioski w Current Context Storage oraz powiązane źródła w RAG sesyjnym,
2. Agent pozytywny wzmacnia najlepiej uzasadnione twierdzenia poprzez dodanie dodatkowych uzasadnień,
3. Agent sceptyczny identyfikuje sprzeczności i braki dowodowe, dopisując je jako nowe wpisy do pamięci,
4. Agent lider dokonuje finalnej selekcji i syntezy na podstawie pełnego stanu pamięci.

Takie podejście można interpretować jako przejście od komunikacji opartej na tekście do komunikacji opartej na **współdzielonym stanie i dowodach**. Oczekiwanym skutkiem jest większa stabilność systemu, lepsza kontrola jakości wniosków oraz możliwość pełnego śledzenia procesu decyzyjnego.

3.2 Tryb współpracy Concurrent

Tryb **Concurrent** stanowi realizację koncepcji asynchronicznej współpracy agentów. W tym modelu trzej agenci techniczni pracują jednocześnie, stale obserwując zmiany w *Current Context Storage*.

W przeciwieństwie do trybu Group Chat, gdzie komunikacja jest turowa, w trybie współbieżnym interakcja zachodzi poprzez zdarzenia (*event-driven*). Gdy *Technical Analyst* zauważy, że *Fundamental Analyst* dodał dane o spadku przychodów, może natychmiast zareagować – dopisując do współdzielonego stanu nowy wpis z adnotacją reinterpretującą trend wzrostowy na wykresie. Operacja ta jest zgodna z zasadą append-only: agent nie modyfikuje cudzego wpisu, lecz tworzy nowy z polem odniesienia do komentowanego rekordu.

Mechanizmy wsparcia w trybie Concurrent są analogiczne do trybu Group Chat i mogą obejmować:

- **Current Context Storage** – pełni rolę magistrali komunikacyjnej; agenci reagują na nowe wpisy innych agentów, co umożliwia emergentną koordynację bez centralnego orkiestratora,
- **RAG sesyjny** – agenci mogą odwoływać się do tych samych fragmentów źródeł ze-

branych przez innych uczestników analizy, co ogranicza redundantne zapytania do zewnętrznych API.

Kwestia spójności zapisu jest rozwiązana przez model append-only: ponieważ każdy wpis jest niezależnym rekordem opatrzonym metadanymi agenta i znacznikiem czasu, jednoczesny zapis wielu agentów nie powoduje konfliktów. **Zaletą tego podejścia** jest wysoka dynamika i zdolność do wyłaniania się wniosków w locie (*emergent reasoning*), co skraca czas potrzebny na osiągnięcie konsensusu w porównaniu do klasycznej, wieloetapowej debaty.

3.3 Tryb współpracy Parallel

Tryb **Parallel** został zaprojektowany jako najczystsza forma niezależnej analizy, mająca na celu całkowite wyeliminowanie uprzedzeń poznawczych (*confirmation bias*, *herd hallucination*) wynikających z wzajemnego wpływu agentów.

Proces w tym trybie dzieli się na dwa odizolowane etapy:

1. **Faza izolowanej analizy:** Trzej agenci techniczni (*Researcher, Technical, Fundamental*) zostają uruchomieni w całkowitej izolacji – każdy z nich ma dostęp do zewnętrznych API, ale nie obserwuje pracy pozostałych. Wyniki ich prac są zapisywane jako niezależne obiekty danych.
2. **Faza agregacji:** Po zakończeniu wszystkich analiz system uruchamia agenta **Reportera (Agregatora)**. Jego zadaniem jest synteza trzech gotowych, niezależnych raportów w jeden spójny dokument.

Mechanizmy wsparcia w trybie Parallel różnią się od poprzednich trybów ze względu na wymóg izolacji:

- **Current Context Storage** – w fazie izolowanej każdy agent operuje na własnej, prywatnej instancji pamięci kontekstowej. Dopiero w fazie agregacji Reporter otrzymuje dostęp do wszystkich trzech instancji jako oddzielnych obiektów wejściowych, co zachowuje pełną separację analiz,
- **RAG sesyjny** – analogicznie, każdy agent buduje własną, izolowaną sesję wektorową. Reporter może opcjonalnie odpytać wszystkie trzy sesje w fazie agregacji w celu weryfikacji twierdzeń zawartych w raportach cząstkowych.

Tryb ten jest bezpośrednią odpowiedzią na problem halucynacji stadnej, która niekiedy występowała w trybie Group Chat. Poprzez wymuszenie niezależności w fazie analitycznej tryb Parallel gwarantuje, że każda z trzech dziedzin analizy zostanie zaprezentowana w finalnym raporcie z równą wagą, co powinno skutkować najwyższym oczekiwanym wynikiem w metryce *Data Richness*.

Bibliografia

[1] E. Jasińska, W. Kurowska i M. Ostaszewski, *Rozbudowa projektu Systemy Multi-Agentowe LLM do Analizy Sprawozdań Finansowych*, Dokumentacja projektu, 2026.

[2] T. Cai i in., "FinDebate: Multi-Agent Collaborative Intelligence for Financial Analysis," *arXiv preprint arXiv:2509.17395*, 2025.

[3] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan i H. D. Nguyen, "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2025.

[4] N. Maruyama, T. Yoshida, H. Sato, A. Masumori i T. Ikegami, "A Concurrent Modular Agent: Framework for Autonomous LLM Agents," *arXiv preprint arXiv:2508.19042*, 2025.

[5] S. Pugachev, "CodeCRDT: Observation-Driven Coordination for Multi-Agent LLM Code Generation," *arXiv preprint arXiv:2510.18893*, 2025.

[6] J. Li, Q. Zhang, Y. Yu, Q. Fu i D. Ye, *More Agents Is All You Need*, 2024. arXiv: 2402.05120 [cs.CL]. URL: https://arxiv.org/abs/2402.05120

[7] Anthropic, *How we built our multi-agent research system*, https://www.anthropic.com/engineering/multi-agent-research-system, Dostęp: 2026-04-04, czerwiec 2025.

[8] E. Zhang, E. Zhu, G. Bansal, A. Fourney, H. Mozannar i J. Gerrits, "Optimizing Sequential Multi-Step Tasks with Parallel LLM Agents," *arXiv preprint arXiv:2507.08944*, 2025.

[9] Y. Xiao, E. Sun, D. Luo i W. Wang, *TradingAgents: Multi-Agents LLM Financial Trading Framework*, 2025. arXiv: 2412.20138 [q-fin.TR]. URL: https://arxiv.org/abs/2412.20138